



Full-Stack Software Development

Back-end: DTOs

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

DTOs – CORS - Postgres

- In this ppt:
 - Data Transfer Objects
 - CORS
 - Postgres database

DTO – Data transfer object

- Why use DTO's
 - **Data Hiding:** DTOs allow you to expose only the necessary data to clients, hiding sensitive information or complex object structures.
 - **Decoupling:** They help separate the internal data model from the external API representation.
 - **Performance:** By transferring only required data, DTOs can improve network performance, especially in distributed systems.
 - **Flexibility:** You can tailor DTOs to specific use cases, combining data from multiple entities or omitting unnecessary fields.

DTO – Data transfer object

- Simple objects with no business logic
- Contain only fields, getters, setters, constructors

=> A perfect fit for records

```
public record ScheduleInput( 8u
    @NotNull Instant start,
    @NotNull Instant end, 1
    @NotNull Long course, 1
    @NotNull Long lecturer
) {
} 
```

DTO – Data transfer object

- define them in the controller layer
- The controller takes a DTO as input
- The validation happens on the DTO
- The DTO is transferred to the service, where the translation is made to one or more models.
- A new DTO is sent back to the controller and ultimately to the client of the call. (optional)

DTO – Data transfer object

```
package be.ucll.se.courses.backend.controller.dto;  
  
import ...  
  
public record UserInput(  
    @NotBlank 2 usages  
    String username,  
    @NotBlank 1 usage  
    String password,  
    @NotBlank 1 usage  
    String firstName,  
    @NotBlank 1 usage  
    String lastName,  
    @Email 1 usage  
    String email) {  
}
```

→ DTOs are defined in a dto package in the controller layer

→ In our application we will be using Input DTOs almost exclusively

→ Validation happens in the DTO

DTO – Data transfer object

```
@PostMapping  
public Lecturer addLecturer(@RequestBody @Valid LecturerInput lecturerInput) {  
    return lecturerService.addLecturer(lecturerInput);  
}
```

DTOs are what the controller will try to deserialize the incoming JSON into

DTOs are passed to the service

```
public Lecturer addLecturer(LecturerInput lecturerInput) {  
    var user = userService.signup(Role.LECTURER, lecturerInput.user());  
    return lecturerRepository.save(new Lecturer(lecturerInput.expertise(), user));  
}
```

DTOs are passed from service to service

To simplify the code we have opted to return domain models to the client instead of mapping to an output or response DTO

Service instantiates Domain objects to execute the call.

DTO – Data transfer object

```
package be.ucll.se.courses.backend.controller.dto;  
  
import ...  
  
public record LecturerInput( 4 usages new *  
    @NotBlank String expertise, 1 usage  
    @Valid UserInput user) {} 1 usage
```

→ DTOs are defined in a dto package in the controller layer

→ DTO can have nested DTOs, but not model classes

→ Nested DTO must have @Valid annotation and have its own validation

CORS

- **Cross-Origin Resource Sharing**
- Browser standard is: Same Origin Policy (SOP), only scripts from the same origin can interact with each other:
 - same scheme (http, https, file, ftp,...)
 - Same domain,
 - And same port number (if the URL includes a port).

=> defence against **Cross Site Request Forgery (CSRF)** attack

CORS

- We'll come back to this when we're tackling security
- For now: add the `@CrossOrigin` annotation to every controller class:
- This allows all api calls from the given url.

```
@RestController  Jeroen Rombouts
@RequestMapping("/courses")
@CrossOrigin(origins = "http://localhost:8080")
public class LecturerController {

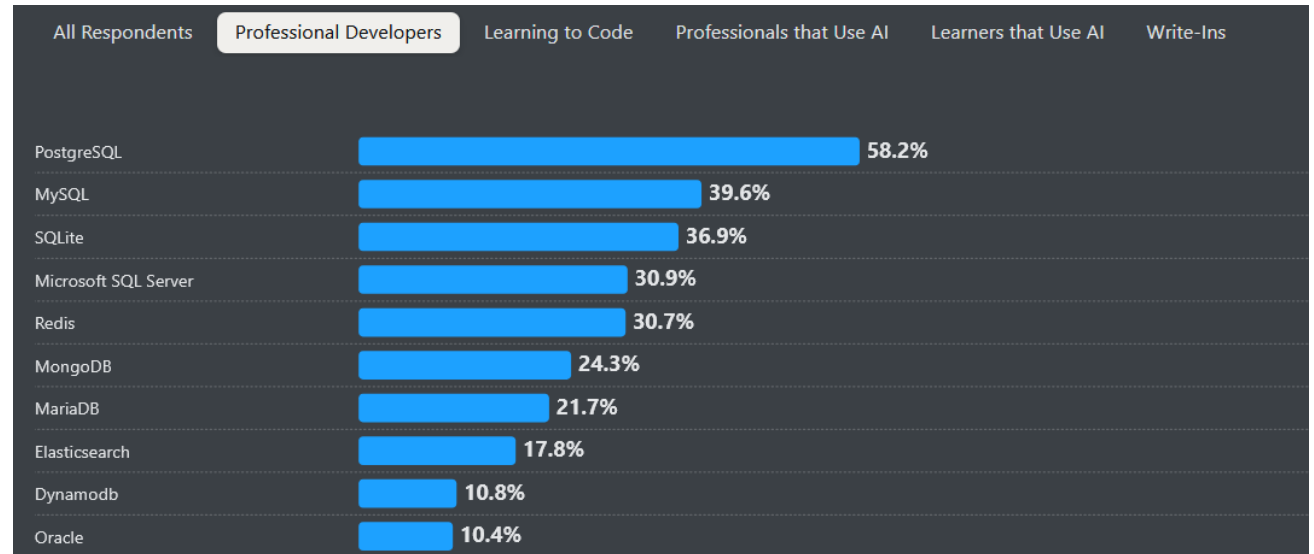
    @GetMapping  Jeroen Rombouts
    public List<Lecturer> getLecturers() {
```

PostgreSQL database

Instead of using an in-memory database, we're going to use a real database

PostgreSQL is becoming an industry standard. In this year's developers' survey 58.2% of all professional developers indicated that they have used PostgreSQL

The installation details are in the lab for this week.



PostgreSQL

- Everything you learned last year works identically for this database, except for the way to connect to it

```
spring.datasource.url=jdbc:postgresql:<url here>  
spring.datasource.username=<username here>  
spring.datasource.password=<password here>
```

- And you need the postgresql dependency in your pom.xml:

```
<dependency>  
  <groupId>org.postgresql</groupId>  
  <artifactId>postgresql</artifactId>  
</dependency>
```

PostgreSQL

- PostgreSQL dialect:
 - “auto-increment” doesn’t exist in the SQL dialect of PostgreSQL
 - Smallserial (smallint), serial(integer) and bigserial(bigint) are used instead:

```
✓ CREATE TABLE OWNER(  
    ID BIGINT PRIMARY KEY AUTO_INCREMENT,  
    NAME VARCHAR(255)  
);
```



```
CREATE TABLE OWNER(  
    ID BIGSERIAL PRIMARY KEY,  
    NAME VARCHAR(255)  
);
```